SOM-based Continuous XGBoost Approach for DDoS Attack Detection and Mitigation in Edge SDN

Shijin Liu*, Hiroaki Fukuda†, Paul Leger‡
*Department of Computer Science and Engineering, Shibaura Institute of Technology, Tokyo, Japan
†Department of Computer Science and Engineering, Shibaura Institute of Technology, Tokyo, Japan
‡Escuela de Ingeniería, Universidad Católica del Norte, Coquimbo, Chile
ma22160@shibaura-it.ac.jp, hiroaki@shibaura-it.ac.jp, pleger@ucn.cl

***Abstract*—Software-Defined Networking (SDN) significantly enhances network programmability and configuration flexibility by separating the control plane from the data plane. However, its centralized control architecture also introduces new security vulnerabilities, making the controller a prime target for Distributed Denial-of-Service (DDoS) attacks. Although machine learning (ML)-based approaches demonstrate outstanding performance in enhancing attack detection accuracy—especially XGBoost, which shows outstanding capabilities in both accuracy and computational overhead, they often face critical challenges: deploying models on controllers or switches can introduce detection latency or significant computational overhead, while models deployed on edge devices suffer from insufficient labeled training data and weak detection capabilities against emerging attack patterns, finally XGBoost does not perform well with small training dataset size. To address these issues, this paper proposes a Self-Organizing Map (SOM)-based continuous XGBoost learning approach (SCX). SCX is deployed on edge devices, enabling real-time model learning and updates using unlabeled network traffic and addressing the performance decline of XGBoost with insufficient training data. Experimental results show that SCX's accuracy is improved to 99.91% and the F1 score is improved to 99.93%, proving SCX can effectively achieve optimal detection performance. Through comparison, SCX outperforms existing mainstream models in detection accuracy.**

***Index Terms*—Software-Defined Networking, Distributed Denial of Service, XGBoost, SOM, Continuous Learning**

I. Introduction

The network is the core component of modern information infrastructure, serving as the critical enabler for sharing information and resources. Traditional network architectures are relatively static and inflexible, making it difficult to reformulate and reconfigure new protocols and rules within the network. Control logic and routing protocols are encapsulated in physical forwarding devices, posing significant challenges for dynamic reconfiguration and configuration adjustments of network protocols [1].

Software-Defined Networking (SDN) fundamentally changed the network management paradigm by separating the control plane from the data plane, improving network programmability and configuration flexibility [2]. However, the centralized control architecture of SDN also introduces new security vulnerabilities. As the core node for network control, the controller is vulnerable to single-point-of-failure (SPOF) risks and becomes a prime target for distributed denial-of-service (DDoS) attacks, severely threatening the availability and robustness of network systems.

DDoS attackers typically control multiple distributed devices serving as bots. They coordinate to generate fraudulent traffic or requests, such as sending massive amounts of ICMP, TCP, or UDP packets, with the intention of exhausting target resources or congesting the victim's network [3].

Various studies have explored methods for addressing DDoS attacks in SDN environments. Recently, machine learning-based approaches have garnered significant attention for their ability to enhance attack detection accuracy [4]. However, concurrent research has also highlighted a critical flaw in such approaches: the limited effectiveness of detection models in identifying and mitigating DDoS attacks, as these models are typically deployed either on the SDN controller or on the switch. Deploying detection models on SDN switches reduces detection latency but normally increases computational overhead on switches, potentially creating performance bottlenecks. Similarly, deploying detection models on SDN controllers centralizes detection efforts, but it also introduces additional detection latency and computational load on the controller. Therefore, a detection mechanism deploying optimized machine learning models on edge devices has been proposed [5]. This novel deployment mechanism completely alleviates the detection burden on SDN controllers and switches. Experimental results demonstrate that the XGBoost [6] model performs most effectively in DDoS detection, outperforming other models.

However, XGBoost's accuracy falls significantly in small training dataset sizes and requires sufficient labeled training data to achieve optimal performance [7]. Obtaining large

amounts of labeled data is challenging in real-world production environments, unlabeled data is often abundant and readily available. Furthermore, diverse edge network environments may continuously generate novel data samples that were not represented in the training dataset. This poses a significant challenge to model generalization capabilities, as the model must identify and handle inputs it has never encountered before. [8]. Machine learning models trained on static datasets often struggle to adapt to new data types. To address XGBoost's accuracy decline with insufficient training data and its generalization challenges, we propose a self-organizing map (SOM)-based continuous XGBoost learning approach (SCX). The core idea of SCX is to generate pseudo labels for unlabeled data by leveraging the SOM clustering algorithm, then continuously train the XGBoost model with the pseudo-labeled data. To ensure SCX can reliably calculate and generate high-quality pseudo labels for unlabeled data, we leverage the unique clustering properties and preserved topological information of SOM to propose a high-quality pseudo label generation algorithm, and utilize these newly labeled data to train and update the XGBoost model in real time, even with a small initial training dataset. Experimental results show that SCX's accuracy is improved to 99.89% and the F1 score is improved to 99.90%. This result demonstrates that SCX has achieved optimal performance.

The subsequent sections of this paper are organized as follows: Section II delves into related works and provides a comprehensive overview of background concepts, including SDN architecture, SOM principles, and the used data sets. Section III describes the mathematical principles and implementation of SCX in detail. Section IV introduces the experimental results and comparison with related works.

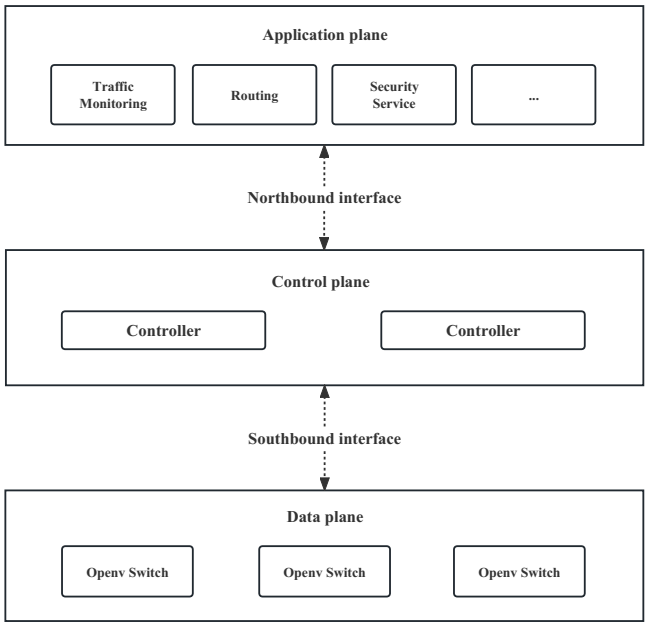

Fig. 1. SDN architectural view

II. Related Works and Background

A. Related Works

DDoS attack detection in SDN environments can generally be categorized into statistical-based methods and machine learning-based methods. Statistical-based methods provide excellent real-time performance, while machine learning-based approaches achieve higher detection accuracy at the cost of computational resources [9]. Considering that realistic DDoS attack detection tasks require maximizing accuracy to minimize false identification for normal connections, machine learning-based methods have garnered increased attention due to their effectiveness.

[10] proposed a DDoS attack detection system based on convolutional neural networks (CNN). The CNN model was trained and evaluated using a customized dataset and deployed on the controller. This model achieved a 99% attack detection accuracy rate. However, several limitations remain to be addressed, such as the limited training records when using real-time traffic as the dataset. Additionally, since the detection model is deployed on the controller, it imposes high overhead on controllers and switches, leading to resource saturation and link overload between controllers and switches.

[11] evaluated many machine learning and deep learning models for Intrusion Detection Systems (IDS) using the UNSW-NB15, NSL-KDD, and CIC-IDS 2017 datasets. Among the tested models, XGBoost consistently achieved the highest accuracy, demonstrating its robustness in handling complex network traffic. Feature selection methods, including Lasso regularization and SelectKBest (ANOVA), were employed to simplify the model, with Lasso maintaining near-optimal performance. Experimental results show that XGBoost with full features achieves an accuracy of 99.89% on the CIC 2017 dataset, while the accuracy after feature selection is 99.86%. However, this paper did not provide further data about CPU and memory usage or other performance overhead metrics. It remains unclear whether XGBoost is suitable for deployment on edge devices.

In [5], machine learning models are deployed at the edge of SDN-IoT networks, enforcing mitigation rules through SDN controllers. Four popular classifiers, K-Nearest Neighbors (K-NN), Random Forest (RF), XGBoost, and Feedforward Neural Network (FFNN) were evaluated on the benchmark datasets CICIDS2017 and Edge-IIoTset. The implementation demonstrates that XGBoost outperforms other models in binary DDoS detection. On the Edge-IIoTset dataset, the test time was 3.559 seconds. By classifying real-time traffic within 3.946 milliseconds while utilizing only 8.80% of memory and a 0.5-second window size, it demonstrates superior performance. This related paper supplements experimental data on performance overhead, proving that XGBoost is optimally suited for deployment on the edge of SDN-IoT networks for real-time traffic classification. However, this related paper does not address countermeasures for scenarios with insufficient labeled data.

Pseudo-labeling serves as a general strategy for addressing the scarcity of labels in semi-supervised learning. However, traditional pseudo-labeling methods introduce substantial noise, leading to degraded model performance [12]. [13] argues that the poor performance of pseudo labels is caused by poorly calibrated models generating erroneous high-confidence predictions; these predictions produce numerous incorrect pseudo labels, leading to noisy training results. Therefore, an uncertainty-aware pseudo label selection (UPS) framework is proposed, which enhances pseudo label accuracy by substantially reducing the amount of noise encountered during training by using MC Dropout approach [14]. The key insight in UPS is that optimal pseudo-label weighting should simultaneously consider two factors: the model's prediction confidence and the data's distribution density. However, this method requires performing T iterations (e.g., T=10 or 100) of random forward propagation for each unlabeled sample. When dealing with massive unlabeled datasets (such as network traffic data), the computational overhead of this approach becomes high and is unsuitable for edge computing environments.

B. Background

1) OpenFlow-based SDN architecture and Proposed System Design: The Software-Defined Networking (SDN) architecture is divided into three layers: the data plane, control plane, and application plane.

- **Data Plane**: Composed of switches, this layer is solely responsible for forwarding packets according to instructions. Its core is the OpenFlow protocol [15], with each switch maintaining flow tables. Each flow table entry includes a Rule, an Action, and a Counter. When a packet arrives, a switch matches it against a rule:
  - **If a match is found**: The corresponding action (e.g., forward, drop) is performed.
  - **If no match is found**: The switch requests a new rule from the controller.
- **Control Plane**: This is the network's "brain," consisting of one or more controllers. It manages the entire network and uses the southbound interface (primarily OpenFlow) to communicate with and send forwarding policies to the switches in the data plane.
- **Application Plane**: This layer hosts various network applications. They use the northbound interface (typically a RESTful API) to interact with the controller, allowing them to retrieve network information and create flow rules that directly influence the packet forwarding process [16].

2) Self-Organizing Map (SOM): The Self-Organizing Map (SOM) is a classic unsupervised neural network [17]. The primary function of SOM is to map high-dimensional input data onto a typically two-dimensional, discrete $n \times n$ grid of neurons while preserving the original topological structure of the input data. This means that data samples that are similar in the original high-dimensional space will be mapped to neurons that are spatially adjacent on the two-dimensional grid.

The learning process of an SOM is a form of Competitive Learning, which primarily involves the following steps:

1) **Initialization:** Each neuron on the grid is assigned a weight vector of the same dimension as the input data, often with random initial values.
2) **Competition:** For each input training sample, the Euclidean distance between it and the weight vector of every neuron is calculated. The neuron with the minimum distance is identified as the "winning neuron" or the *Best Matching Unit (BMU)*.
3) **Cooperation:** The weight vectors of the winning neuron and its neighboring neurons on the grid are updated according to a neighborhood function, moving them closer to the input sample.
4) **Iteration:** The preceding steps are repeated until the network converges. During this process, the learning rate and the neighborhood radius are gradually decreased.

3) Extreme Gradient Boosting (XGBoost): XGBoost is a sequential learning method based on decision trees and boosting. By iteratively combining multiple decision tree classifiers, XGBoost achieves high accuracy while minimizing false positives, thereby effectively optimizing resource utilization. As a supervised learning algorithm, XGBoost constructs a predictive model by integrating multiple weak learners (typically decision trees) [6]. In each iteration, a new decision tree is trained to fit the residuals (i.e., the error between predicted and actual values) of the ensemble model formed by all trees from the previous round. In this way, the model is progressively optimized along the gradient of the loss function, continuously reducing the overall prediction error.

However, although XGBoost has been proven suitable for deployment in edge device detection systems in terms of computational overhead, detection speed, and accuracy, it still faces performance decline issues on small training datasets [18]–[20].

4) Dataset: Many datasets are utilized in the evaluation of DDoS attack defense. The widely used KDD 99 and NSL-KDD datasets mostly concentrate on conventional DDoS attacks. The CIC DoS 2019 dataset not only contains regular high-rate DDoS attack data but also includes reflective and exploitive DDoS attacks. In addition, a large number of low-rate DDoS attacks were captured in the CIC DoS 2017 dataset [21].

This paper primarily focuses on the Friday data record in the CIC 2017 datasets because they contain a large number of attack types and precomputed features, and DDoS records. Furthermore, to prevent the model from erroneously memorizing IP addresses and port numbers as attack features, we removed all features related to IP addresses and ports from the CIC dataset to achieve IP-agnostic, ultimately retaining 78 features.

III. Methodology

This section describes the detection and mitigation framework and the implementation principles and execution steps of SCX.

A. Detection and Mitigation System Framework Design

Figure 2 shows the proposed edge SDN DDoS detection and mitigation framework. The detection approach SCX is deployed on edge devices. When attack traffic is detected, SCX sends a signal containing attacker identification information (such as IP address) to the controller. The controller locates the attacker based on the signal and issues drop rules to cut off the connection.

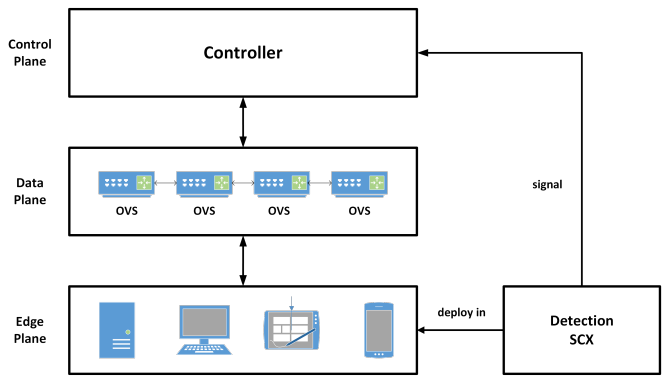

Fig. 2. DDoS detection and mitigation system design

B. SCX Design Principle

Based on the preceding discussion, we contend that the core contradiction of existing DDoS edge detection systems lies in the conflict between the difficulty of obtaining large amounts of labeled data and the accuracy decline of XGBoost when training data is insufficient. To address this problem, our core approach is employing a clustering model to analyze unlabeled data and generate high-quality pseudo label for them. Then, these high-quality pseudo label data can be utilized as training data for XGBoost, thereby resolving its data scarcity problem.

Compared to other clustering algorithms, SOM has an unparalleled advantage in preserving more topological information during the clustering and dimensionality reduction process. Specifically, K-Means uses the mean value as the cluster center which is sensitive to outlier noisy points; extreme outliers can significantly deviate the cluster center position. SOM avoids excessive grid shift toward outliers by introducing neighborhood functions and quantifies sample reliability through calculating quantization error (QE).

During the subsequent confidence-based pseudo-label generation phase we propose, samples with high QE will be assigned low confidence. This means that SOM can output labels with continuous values for data samples located on fuzzy boundaries between classes, rather than discrete values as in traditional algorithms like K-Means [22]. Based on the aforementioned advantages, we modified the SOM and adopted it as the clustering algorithm for SCX. Due to SOM's powerful capabilities in unsupervised clustering and data visualization, SOM is exceptionally well-suited for exploring the intrinsic structure and distribution of complex datasets without label guidance. This is the core reason we selected it to process unlabeled network traffic and generate high-quality pseudo-labels.

Furthermore, according to XGBoost's strong performance, low computational overhead, and adaptability to edge devices, we employ XGBoost as a classifier and optimization target of SCX. XGBoost continuously receives data labeled by SOM and combines all previous data into a unified training dataset for continuous learning and model updates. The general workflow is as follows:

- Train the initial clustering model SOM and the classification model XGBoost using a small training dataset.
- Label neurons using majority voting based on the distribution of data labels that match the SOM neurons.
- When unlabeled data is input to the SOM, determine whether to label it based on the matched neuron and filtering algorithm.
- Package the labeled data and transmit them to XGBoost to update the classification model.

C. SCX Detection Approach

1) Phase 1: Offline Model Initialization: This phase utilizes a small, initially labeled dataset $(X_{\text{initial}}, y_{\text{initial}})$ to establish the system's baseline capabilities.

① **Data Preprocessing:** Input feature vectors (Featured data) $x \in X_{\text{initial}}$ are standardized to eliminate the influence of different feature scales. This processing ensures the fairness and validity of subsequent Euclidean distance-based topology learning. Each feature dimension is transformed according to the formula:

$$x' = \frac{x - \mu}{\sigma} \tag{1}$$

where μ and σ are the mean and standard deviation of the feature on the training set, respectively.

② **Unsupervised Topology Learning via SOM:** A 2D SOM grid is initialized. For each input vector $x(t)$ from the training set, the SOM updates its weights through a competitive learning process. First, the BMU, denoted as c, is identified. The BMU is the neuron whose weight vector w_c has the minimum Euclidean distance to the input vector $x(t)$:

$$c = \arg\min_i \|x(t) - w_i(t)\|_2 \tag{2}$$

Subsequently, the weight vectors of the BMU and its neighboring neurons are updated to move closer to the input vector, following the rule:

$$w_i(t+1) = w_i(t) + h_{ci}(t) \cdot [x(t) - w_i(t)] \tag{3}$$

where $h_{ci}(t)$ is the neighborhood function that determines the strength of the update, typically a Gaussian function that decays over time:

$$h_{ci}(t) = \alpha(t) \cdot \exp\left(-\frac{\|r_c - r_i\|^2}{2\sigma^2(t)}\right) \tag{4}$$

Here, $\alpha(t)$ is the learning rate, $\sigma(t)$ is the neighborhood radius, and r_i and r_c are the coordinates of neuron i and

the BMU c on the grid, respectively. Both $\alpha(t)$ and $\sigma(t)$ decrease monotonically with the number of iterations t.

③ **Probabilistic Neuron Labeling:** After the SOM is trained, the initial labeled dataset X_{initial} is mapped onto the grid again. For each neuron i in the grid, we construct a set S_i containing all training samples for which i is the BMU. The final label L_i of neuron i is determined by a majority vote of the labels of the samples in S_i:

$$L_i = \arg\max_{k \in \{0,1\}} \sum_{x_j \in S_i} \mathbb{I}(y_j = k) \tag{5}$$

where $k = 0$ represents normal and $k = 1$ represents attack, and $\mathbb{I}(\cdot)$ is the indicator function. Concurrently, we compute the confidence of this label, $Conf_{\text{neuron}}(i)$, which is the proportion of the majority class:

$$Conf_{\text{neuron}}(i) = \frac{\max_{k \in \{0,1\}} \sum_{x_j \in S_i} \mathbb{I}(y_j = k)}{|S_i|} \tag{6}$$

④ **Initial Classifier Training:** The same initial dataset $(X_{\text{initial}}, y_{\text{initial}})$ is used to train a baseline XGBoost model. XGBoost iteratively trains a series of decision trees by minimizing a regularized objective function:

$$\mathcal{L}(\phi) = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k) \tag{7}$$

where l is the loss function, $\hat{y}_i$ is the predicted value, and $\Omega(f_k)$ is a regularization term on the complexity of the k-th tree.

2) Phase 2: Online Continuous Learning and Adaptation: This phase is the core of the SCX, responsible for processing unlabeled data streams and enabling the model's self-evolution. The objective of this phase is to address challenges in continuous learning: how to safely learn from unlabeled data streams without compromising model stability.

① **High-Fidelity Pseudo-Label Generation:** For each new unlabeled sample x_u, SCX will generate a pseudo-label. This process is based on a core principle: a reliable pseudo-label must satisfy both group similarity and individual typicality.

- First, identify the BMU c for the new sample x_u via SOM. The neuron's label L_c and confidence score $Conf_{\text{neuron}}(c)$ provide a measure of "group similarity".
- Next, a novel unknown noise attack may be incorrectly mapped to the nearest "normal" label neuron, despite being a poor match. To quantify this "matching quality," we introduce "individual typicality" and employ quantization error (QE) as its metric. QE ($QE(x_u) = \|x_u - w_c\|^2$) measures the distance between a sample and the center of its assigned cluster (neuron weights). A low QE value indicates that x_u is a typical member of its cluster; a high QE value suggests it is an outlier.
- Finally, We propose a composite final confidence score $Conf_{\text{final}}$ (Formula 8) to integrate these two information dimensions. We employ a multiplicative relationship to ensure a sample receives high confidence only when its group is highly pure ($Conf_{\text{neuron}}$ is high) and the sample itself is a typical member of that group (low QE, thus high $1/(1 + \alpha \cdot QE)$).

$$Conf_{\text{final}}(x_u) = Conf_{\text{neuron}}(c) \times \frac{1}{1 + \alpha \cdot QE(x_u)} \tag{8}$$

where α is a hyperparameter that modulates the impact of the QE. This formula ensures that a high confidence score is assigned only when a sample maps to a neuron that is both highly pure ($Conf_{\text{neuron}}$ is high) and a close match (QE is small).

② **Dual-Criteria Pseudo-Label Filtering:** To further ensure model stability and mitigate the risk of error amplification, we have implemented a dual-filter mechanism.

- **Confidence Thresholding:** This is a static fidelity check ($Conf_{\text{final}} > \theta_{\text{conf}}$), ensuring only samples meeting high confidence thresholds proceed to the next step.
- **Outlier Filtering:** This is a dynamic, batch-based anomaly detection mechanism ($QE(x_u) \leq \text{Percentile}(QE_{\text{batch}}, 80)$). It addresses the issue where bursts of noise or data influxes can cause the overall QE of an entire batch to be abnormally high. This filter rejects the "least typical" 20% of samples in the current batch, even if their final confidence scores may have met the threshold. The QE threshold of 80 was experimentally determined on our dataset as the optimal balance point, striking equilibrium between "noise elimination" (stability) and "learning new patterns" (generalization capability).

This dual criterion effectively filters out samples that might be outliers or noise, even if they fall within a specific cluster.

③ **Incremental Model Retraining:** The high-quality pseudo-labeled samples that pass the filtering stage are added to a growing training data buffer. When a sufficient number of new samples have been accumulated in the buffer, SCX re-trains a new version of the XGBoost model (e.g., v1.1, v1.2...) using the entire buffer (which includes the initial data and all accumulated pseudo-labeled data). This iterative cycle of "**collect-filter-retrain**" allows the model to continuously absorb new knowledge and adjust its decision boundary according to the current network environment, thereby effectively combating concept drift.

IV. Experiment and Evaluation

To evaluate the feasibility and performance of SCX, we have done some experiments. This section introduces the performance metrics used for evaluation, the experimental results, and comparisons with related research.

A. Metrics

Common metrics used for evaluating the effectiveness of machine learning algorithms include Accuracy, Precision, Recall, and F1 Score [23]. The degree of machine learning

classification approaches can be evaluated using accuracy. Metrics are defined as the following equations (from equation 9 to 12). Among these metrics, TN stands for True Negative and refers to instances where the model correctly identified the negative class. Instances where the model correctly identified the positive class are referred to as True Positives (TP). False Negative (FN) cases are those in which the model predicted a negative class, but the actual class was positive. Instances where the model predicted the positive class but the actual class was negative are referred to as False Positives (FP) [24].

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN} \tag{9}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{10}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{11}$$

$$\text{F1 score} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} \tag{12}$$

B. Result and Discussion

The experimental testbed is built on a host running Windows 11, CPU is Core U9-275HX with 32GB RAM. The total size of the experimental data is 300,000, with 10% used for initial model training, 25% for testing, and 65% the labels were erased for the continuous learning phase.

Figure 3 shows the accuracy and F1 score over 10 iteration cycles. The accuracy improved from 99.89% in the initial model to 99.92%, while the F1 score increased from 99.91% to 99.93%. Both metrics achieved the best performance reported in [11]. These two metrics show a general upward trend, indicating that SCX can continuously update and improve itself. This result demonstrates the effectiveness of the pseudo-label generation algorithm and the DDoS attack detection algorithm in SCX.

Figure 4 shows the precision and recall over 10 iteration cycles. Figure 5 shows the number of samples labeled with pseudo labels in each round, with the x-axis representing the iteration round and the y-axis representing the number of samples. These figures demonstrate that precision and recall maintain high performance even as the number of pseudo-labeled data samples increases.

Table I shows SCX's usage of computational resources, including CPU utilization, memory usage, and classification time. And including scenarios where the number of cores used is limited to simulate verification results under different performance platforms. SCX is developed entirely on a CPU-based computing platform, requiring no GPU support. SCX's memory usage and classification speed sufficiently demonstrate its suitability for deployment on edge devices.

In a continuous learning environment, if an algorithm incorrectly labels unlabeled data as pseudo labels (i.e., high-noise labels) and uses them for retraining, the model's performance will degrade. This leads to generating more erroneous labels in the future, forming a negative feedback loop. However, the results of our experiment show the opposite trend. Over eight iteration cycles, the number of samples assigned pseudo labels and used for retraining steadily increased.

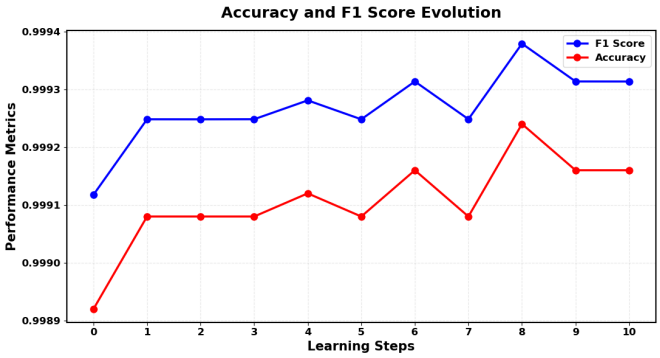

Fig. 3. Accuracy and F1 score over 10 iteration cycles

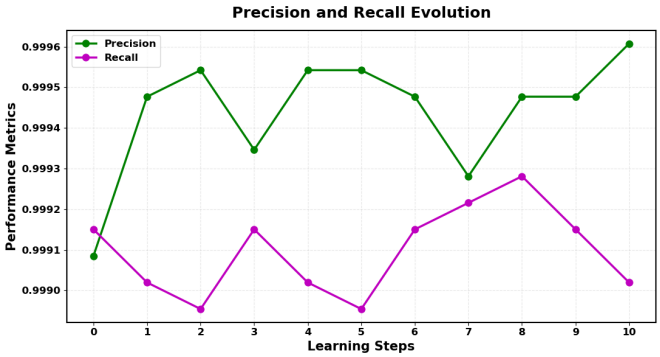

Fig. 4. Precision and recall over 10 iteration cycles

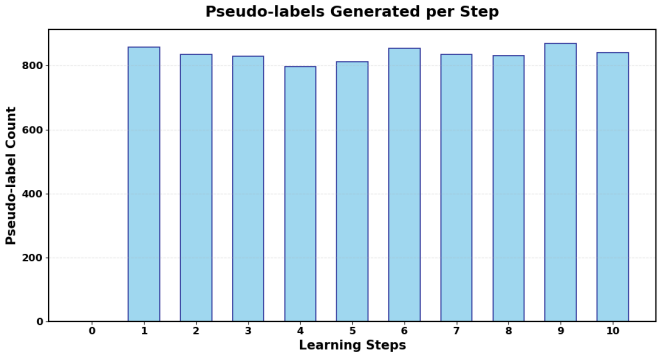

Fig. 5. Number of samples pseudo-labeled within each iteration cycle

Concurrently, the model's precision and recall remained consistently at high and stable levels, showing no decline despite the addition of new data. These results provide strong proof of the high accuracy of SCX's pseudo-label generation algorithm. It successfully filters out low-noise, high-quality samples and utilizes this high -quality new data for iterative training, thereby preventing performance degradation and

achieving further improvements in overall accuracy and F1 scores.

TABLE I
CPU UTILIZATION, MEMORY USAGE AND CLASSIFICATION TIME

CPU Core	CPU Utilization	Memory Usage	Classification Time
1	6.1%	456.29 MB	0.532 S
2	12.5%	458.51 MB	0.462 S
8	23.3%	458.60 MB	0.370 S
16	36.4%	459.89 MB	0.406 S
Full	56.6%	456.13 MB	0.312 S

TABLE II
PERFORMANCE COMPARISON WITH OTHER ALGORITHMS

Algorithm	Accuracy
Learning without Forgetting	88.35%
Experience Replay	98.95%
Dark Experience Replay	99.46%
XGBoost	99.90%
SCX(Our)	99.91%

C. Comparison

[25] evaluated three major continuous learning algorithms: Learning without Forgetting, Experience Replay, and Dark Experience Replay on the CIC2017 dataset. Their experiments demonstrated that DER achieves the optimal balance between new and old knowledge when handling the complex feature distribution of the CIC dataset. We also compared the normal XGBoost algorithm evaluated in [26]. We adopted the models proposed in this paper as the comparative baseline. As shown in Table II, SCX's accuracy significantly outperformed existing mainstream models.

V. CONCLUSION

To address the accuracy decline caused by the scarcity of labeled data in DDoS detection tasks using XGBoost within edge SDN environments, we innovatively propose SCX—a high-quality pseudo- label generation algorithm based on Self-Organizing Maps (SOM). This enables continuous incremental training of the XGBoost model to enhance detection accuracy. Experiments validate SCX's effectiveness, with accuracy and F1 score reaching 99.89% and 99.90% respectively, in continuous learning. This proves SCX is an effective, self-evolving detection approach capable of adapting to dynamic network environments despite insufficient pre-training data.

Our future research will focus on two key optimizations. First, we will work on systematic feature selection across the entire SCX model. This aims to further reduce computational overhead and eliminate noise. Secondly, we will focus on model lightweighting to ensure the SCX is suitable for various resource-constrained hardware, particularly SDN edge devices.

REFERENCES

[1] R. Swami, M. Dave, and V. Ranga, "Software-defined networking-based ddos defense mechanisms," *ACM Computing Surveys (CSUR)*, vol. 52, no. 2, pp. 1–36, 2019.

[2] N. McKeown, "How sdn will shape networking," *Open Networking Summit*, 2011.

[3] C. Douligeris and A. Mitrokotsa, "Ddos attacks and defense mechanisms: classification and state-of-the-art," *Computer networks*, vol. 44, no. 5, pp. 643–666, 2004.

[4] M. Zang, E. O. Zaballa, and L. Dittmann, "Sdn-based in-band ddos detection using ensemble learning algorithm on iot edge," in *2022 25th Conference on Innovation in Clouds, Internet and Networks (ICIN)*, 2022, pp. 111–115.

[5] H. M. Belachew, M. Y. Beyene, A. B. Desta, B. T. Alemu, S. S. Musa, and A. J. Muhammed, "Design a robust ddos attack detection and mitigation scheme in sdn-edge-iot by leveraging machine learning," *IEEE Access*, vol. 13, pp. 10 194–10 214, 2025.

[6] T. Chen, T. He, M. Benesty, V. Khotilovich, Y. Tang, H. Cho, K. Chen, R. Mitchell, I. Cano, T. Zhou *et al.*, "Xgboost: extreme gradient boosting," *R package version 0.4-2*, vol. 1, no. 4, pp. 1–4, 2015.

[7] M. Zou, W.-G. Jiang, Q.-H. Qin, Y.-C. Liu, and M.-L. Li, "Optimized xgboost model with small dataset for predicting relative density of ti-6al-4v parts manufactured by selective laser melting," *Materials*, vol. 15, no. 15, p. 5298, 2022.

[8] A. A. Alashhab, M. S. Zahid, B. Isyaku, A. A. Elnour, W. Nagmeldin, A. Abdelmaboud, T. A. A. Abdullah, and U. D. Maiwada, "Enhancing ddos attack detection and mitigation in sdn using an ensemble online machine learning model," *IEEE Access*, vol. 12, pp. 51 630–51 649, 2024.

[9] Z. Long and W. Jinsong, "A hybrid method of entropy and ssae-svm based ddos detection and mitigation mechanism in sdn," *Computers & Security*, vol. 115, p. 102604, 2022.

[10] J. Wang, Y. Liu, W. Su, and H. Feng, "A ddos attack detection based on deep learning in software-defined internet of things," in *2020 IEEE 92nd Vehicular Technology Conference (VTC2020-Fall)*, 2020, pp. 1–5.

[11] P. Ferreira, E. Martins, J. Silva, and P. Teixeira, "Feature selection and xgboost for enhanced intrusion detection: A comparative study across benchmark datasets," in *2025 13th International Symposium on Digital Forensics and Security (ISDFS)*, 2025, pp. 1–6.

[12] B. Wang, J. Li, Y. Liu, J. Cheng, Y. Rong, W. Wang, and F. Tsung, "Deep insights into noisy pseudo labeling on graph data," *Advances in Neural Information Processing Systems*, vol. 36, pp. 76 214–76 228, 2023.

[13] M. N. Rizve, K. Duarte, Y. S. Rawat, and M. Shah, "In defense of pseudo-labeling: An uncertainty-aware pseudo-label selection framework for semi-supervised learning," *arXiv preprint arXiv:2101.06329*, 2021.

[14] Y. Gal and Z. Ghahramani, "Dropout as a bayesian approximation: Representing model uncertainty in deep learning," in *international conference on machine learning*. PMLR, 2016, pp. 1050–1059.

[15] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "Openflow: enabling innovation in campus networks," *ACM SIGCOMM computer communication review*, vol. 38, no. 2, pp. 69–74, 2008.

[16] C. Banse and S. Rangarajan, "A secure northbound interface for sdn applications," in *2015 IEEE Trustcom/BigDataSE/ISPA*, vol. 1. IEEE, 2015, pp. 834–839.

[17] T. Kohonen, "The self-organizing map," *Proceedings of the IEEE*, vol. 78, no. 9, pp. 1464–1480, 1990.

[18] X. Wang and S. Chang, "Enhancing body fat prediction with wgan-gp data augmentation and xgboost algorithm," *Science Progress*, vol. 108, no. 3, p. 00368504251366850, 2025.

[19] L. M. Kamdem and E. Fokoue, "A computational exploration of emerging methods of variable importance estimation," *arXiv preprint arXiv:2208.03373*, 2022.

[20] T. Dong, I. B. Oronti, S. Sinha, A. Freitas, B. Zhai, J. Chan, D. P. Fudulu, M. Caputo, and G. D. Angelini, "Enhancing cardiovascular risk prediction: development of an advanced xgboost model with hospital-level random effects," *Bioengineering*, vol. 11, no. 10, p. 1039, 2024.

[21] CIC Dos Dataset 2017. [Online]. Available: https://www.unb.ca/cic/datasets/index.html

[22] A. Flexer, "On the use of self-organizing maps for clustering and visualization," *Intelligent Data Analysis*, vol. 5, no. 5, pp. 373–384, 2001.

[23] S. Liu, H. Fukuda, and P. Leger, "Real-time ddos attack defense system in sdn using lssom," in *2023 26th Conference on Innovation in Clouds, Internet and Networks and Workshops (ICIN)*. IEEE, 2023, pp. 69–73.

[24] J. Pei, Y. Chen, and W. Ji, "A ddos attack detection method based on machine learning," in *Journal of Physics: Conference Series*, vol. 1237, no. 3. IOP Publishing, 2019, p. 032040.

[25] S. Prasath, K. Sethi, D. Mohanty, P. Bera, and S. R. Samantaray, "Analysis of continual learning models for intrusion detection system," *IEEE Access*, vol. 10, pp. 121 444–121 464, 2022.

[26] B. Li, X. Lin, and X. Lu, "The performance comparison of some methods based on machine learning and deep learning for the web attack detection," in *International Conference on Electronic Information Engineering and Computer Technology (EIECT 2021)*, vol. 12087. SPIE, 2021, pp. 219–225.